Practical 8

Aim - 1:

Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Code:

```java
import java.util.Scanner;

// Custom checked exception
class DivideByZeroException extends Exception {

    public DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator {

    // Calculator method
    static double calculate(double a, double b, char operator)
            throws DivideByZeroException {

        switch (operator) {

            case '+':
                return a + b;

            case '-':
                return a - b;

            case '*':
                return a * b;

            case '/':
                if (b == 0) {
                    throw new DivideByZeroException(
                            "Cannot divide by zero."
                    );
                }
                return a / b;
```

CEUE20325IT103

```
            default:
                throw new IllegalArgumentException(
                        "Invalid operator."
                );
            }
        }

        public static void main(String[] args) {

            Scanner sc = new Scanner(System.in);

            System.out.println("=== Guarded Calculator ===");
            System.out.println("Enter 'q' to quit.");

            while (true) {

                try {

                    System.out.print("\nEnter first number: ");
                    String input = sc.next();

                    if (input.equalsIgnoreCase("q")) {
                        break;
                    }

                    double a = Double.parseDouble(input);

                    System.out.print("Enter operator (+, -, *, /): ");
                    char operator = sc.next().charAt(0);
```

```
                    System.out.print("Enter operator (+, -, *, /): ");
                    char operator = sc.next().charAt(0);

                    System.out.print("Enter second number: ");
                    double b = sc.nextDouble();

                    double result = calculate(a, b, operator);

                    System.out.println("Result = " + result);

                }

                catch (DivideByZeroException e) {

                    System.out.println(
                            "Error: " + e.getMessage()
                    );

                }

                catch (NumberFormatException e) {

                    System.out.println(
                            "Error: Invalid number."
                    );

                }

                catch (IllegalArgumentException e) {

                    System.out.println(
                            "Error: " + e.getMessage()
```

```
                System.out.println(
                        "Error: Invalid number."
                );

            }

            catch (IllegalArgumentException e) {

                System.out.println(
                        "Error: " + e.getMessage()
                );

            }

            catch (Exception e) {

                System.out.println(
                        "Error: Invalid input."
                );

                sc.nextLine();
            }
        }

        sc.close();

        System.out.println("\nCalculator closed.");
    }
}
```

Output:

```
=== Guarded Calculator ===
Enter 'q' to quit.

Enter first number: 5
Enter operator (+, -, *, /): +
Enter second number: 4
Result = 9.0

Enter first number: 4
Enter operator (+, -, *, /): *
Enter second number: 5
Result = 20.0
```

Learning Outcomes:

After completing this **Guarded Calculator using Java Exception Handling**, students will be able to:

1. **Understand exception handling** using `try`, `catch`, and `throws`.
2. **Create and use custom checked exceptions** such as `DivideByZeroException`.
3. **Handle different types of exceptions** like `NumberFormatException` and `IllegalArgumentException`.
4. **Use the `throw` statement** to generate exceptions when an error condition occurs.
5. **Perform arithmetic operations** using `switch` statements.
6. **Validate user input** and handle invalid inputs safely.
7. **Understand the difference between checked and unchecked exceptions** in Java.
8. **Build a robust calculator program** that continues running even when an input error occurs.

Aim – 2:

Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.

Code:

CEUE20325IT103

```
                "Item '" + item + "' does not exist."
            );
        }

        if (qty <= 0) {

            throw new OutOfStockException(
                    message: message: "Quantity must be greater than zero."
            );
        }

        int available = stock.get(item);

        if (qty > available) {

            throw new OutOfStockException(
                    "Not enough stock for " + item +
                    ". Available = " + available
            );
        }

        stock.put(item, available - qty);

        System.out.println(
                "Issued " + qty + " " + item +
                "(s) successfully."
        );

        System.out.println(
                "Remaining stock = " +
                stock.get(item)
        );
```

CEUE20325IT103

```
        );
    }

    // Display stock
    public void displayStock() {

        System.out.println(x: "\nCurrent Warehouse Stock:");

        for (Map.Entry<String, Integer> entry
                : stock.entrySet()) {

            System.out.println(
                    entry.getKey() + " : " +
                    entry.getValue()
            );
        }
    }
}

public class WarehouseStock {

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Warehouse warehouse = new Warehouse();

        System.out.println(x: "=== Warehouse Stock System ===");

        warehouse.displayStock();
```

```
        System.out.print(
                s: "\nEnter number of requests: "
        );

        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {

            System.out.println("\nRequest " + i);

            System.out.print(s: "Enter item: ");
            String item = sc.next();

            System.out.print(s: "Enter quantity: ");
            int qty = sc.nextInt();

            try {

                warehouse.issue(item, qty);

            }

            catch (OutOfStockException e) {

                System.out.println(
                        "Request failed: " +
                        e.getMessage()
                );

            }
```

```
                System.out.println(
                        x: "Processing continues..."
                );
            }

            warehouse.displayStock();

            sc.close();
        }
    }
```

Output:

```
=== Warehouse Stock System ===

Current Warehouse Stock:
Laptop : 5
Monitor : 3
Mouse : 10
Keyboard : 7

Enter number of requests: 2

Request 1
Enter item: Laptop
Enter quantity: 2
Issued 2 Laptop(s) successfully.
Remaining stock = 3
Processing continues...

Request 2
Enter item: Keyboard
Enter quantity: 4
Issued 4 Keyboard(s) successfully.
Remaining stock = 3
Processing continues...

Current Warehouse Stock:
Laptop : 3
Monitor : 3
Mouse : 10
Keyboard : 3


...Program finished with exit code 0
Press ENTER to exit console.
```

Learning Outcomes:

After completing this **Warehouse Stock Management System**, students will be able to:

1. **Understand custom checked exceptions** by creating and using `OutOfStockException`.
2. **Use `try-catch` blocks** to handle exceptions without stopping the program.
3. **Use `HashMap`** to store and manage item names and their stock quantities.
4. **Perform stock operations** such as issuing items and updating remaining stock.
5. **Validate user input** for invalid items and quantities.
6. **Use classes and objects** to implement a simple warehouse management system.
7. **Understand exception propagation** using the `throws` keyword.
8. **Continue program execution after an exception** using proper exception handling.

Aim – 3:

(For Advanced Learners – home) Create an AutoCloseable resource and use try-with- resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.

Code:

```java
// Custom resource
class MyResource implements AutoCloseable {

    public MyResource() {

        System.out.println(x: "Resource opened.");
    }

    public void use() throws Exception {

        System.out.println(x: "Using resource...");

        throw new Exception(
                message: "Exception occurred while using resource."
        );
    }

    @Override
    public void close() throws Exception {

        System.out.println(x: "Closing resource...");

        throw new Exception(
                message: "Exception occurred while closing resource."
        );
    }
}

public class AutoCloseableDemo {

    Run | Debug
    public static void main(String[] args) {

        try (MyResource resource = new MyResource()) {
```

CEUE20325IT103

```
            try (MyResource resource = new MyResource()) {

                resource.use();

            }

            catch (Exception e) {

                System.out.println(
                        x: "\nMain Exception:"
                );

                System.out.println(
                        e.getMessage()
                );

                System.out.println(
                        x: "\nSuppressed Exceptions:"
                );

                Throwable[] suppressed =
                        e.getSuppressed();

                for (Throwable s : suppressed) {

                    System.out.println(
                            s.getMessage()
                    );
                }
            }
        }
    }
```

Output:

```
Resource opened.
Using resource...
Closing resource...

Main Exception:
Exception occurred while using resource.

Suppressed Exceptions:
Exception occurred while closing resource.


...Program finished with exit code 0
Press ENTER to exit console.
```

CEUE20325IT103

Learning Outcomes:

After completing this **AutoCloseable and Try-with-Resources** program, students will be able to:

1. **Understand the `AutoCloseable` interface** in Java.
2. **Implement automatic resource closing** using `try-with-resources`.
3. **Handle exceptions** using `try-catch`.
4. **Understand suppressed exceptions** that occur while closing a resource.
5. **Use the `getSuppressed()` method** to retrieve suppressed exceptions.
6. **Understand resource management** and why resources should be properly closed.
7. **Create custom resources** by implementing the `AutoCloseable` interface.